MINISTERUL EDUCAȚIEI

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

TERMOSTAT IOT

PROIECT SEMESTRIAL

Autor: **Miruna BÎRZOEȘ**

Conducător științific: **Titlu. ing. Ioan Valentin SITA**

2025

Cuprins

1 INTRODUCERE 2

1.1 CONTEXT GENERAL 2
1.2 OBIECTIVE 2
1.3 SPECIFICAȚII 2

2 STUDIU BIBLIOGRAFIC 5

3 ANALIZĂ, PROIECTARE, IMPLEMENTARE 7

4 CONCLUZII 18

4.1 REZULTATE OBȚINUTE 18
4.2 DIRECȚII DE DEZVOLTARE 18

5 REGULI DE FORMATARE ..... ERROR! BOOKMARK NOT DEFINED.

5.1 FORMATAREA PAGINII ..... **ERROR! BOOKMARK NOT DEFINED.**
5.2 TITLURI ȘI STILURI ..... **ERROR! BOOKMARK NOT DEFINED.**
5.3 FIGURI, TABELE ȘI ECUAȚII ..... **ERROR! BOOKMARK NOT DEFINED.**
*5.3.1 Figuri ..... **Error! Bookmark not defined.***
5.4 TABELE ..... **ERROR! BOOKMARK NOT DEFINED.**
5.5 ECUAȚII ..... **ERROR! BOOKMARK NOT DEFINED.**
5.6 REFERINȚE BIBLIOGRAFICE ..... **ERROR! BOOKMARK NOT DEFINED.**

6 BIBLIOGRAFIE ..... ERROR! BOOKMARK NOT DEFINED.

1 Introducere

1.1 Context general

În contextul actual al revoluției industriale 4.0 și al digitalizării accelerate, conceptul de "Internet of Things" (IoT) a devenit un pilon fundamental în dezvoltarea tehnologică modernă. Interconectarea dispozitivelor fizice cu infrastructura internetului permite colectarea, analiza și utilizarea datelor pentru a optimiza procese, a spori confortul și a eficientiza consumul de resurse.

Monitorizarea parametrilor ambientali (temperatură, umiditate, presiune atmosferică) reprezintă una dintre cele mai răspândite aplicații IoT, având utilitate directă atât în mediul industrial (monitorizarea serverelor, a depozitelor), cât și în cel rezidențial (conceptul de "Smart Home"). Sistemele tradiționale de monitorizare oferă informația doar local, necesitând prezența fizică a operatorului. În contrast, sistemele moderne propun arhitecturi distribuite, unde datele sunt accesibile de oriunde, în timp real, prin intermediul platformelor Cloud și al dispozitivelor mobile.

Această lucrare propune implementarea unui sistem complet de monitorizare, care integrează achiziția de date la nivel hardware cu procesarea în Cloud și vizualizarea pe platforme mobile, demonstrând interoperabilitatea dintre tehnologiile embedded și cele de dezvoltare software de nivel înalt.

1.2 Obiective

Obiectivul general al proiectului este proiectarea, realizarea și testarea unui **Termostat Inteligent IoT** capabil să monitorizeze temperatura și presiunea atmosferică și să ofere acces la aceste date atât local, cât și la distanță.

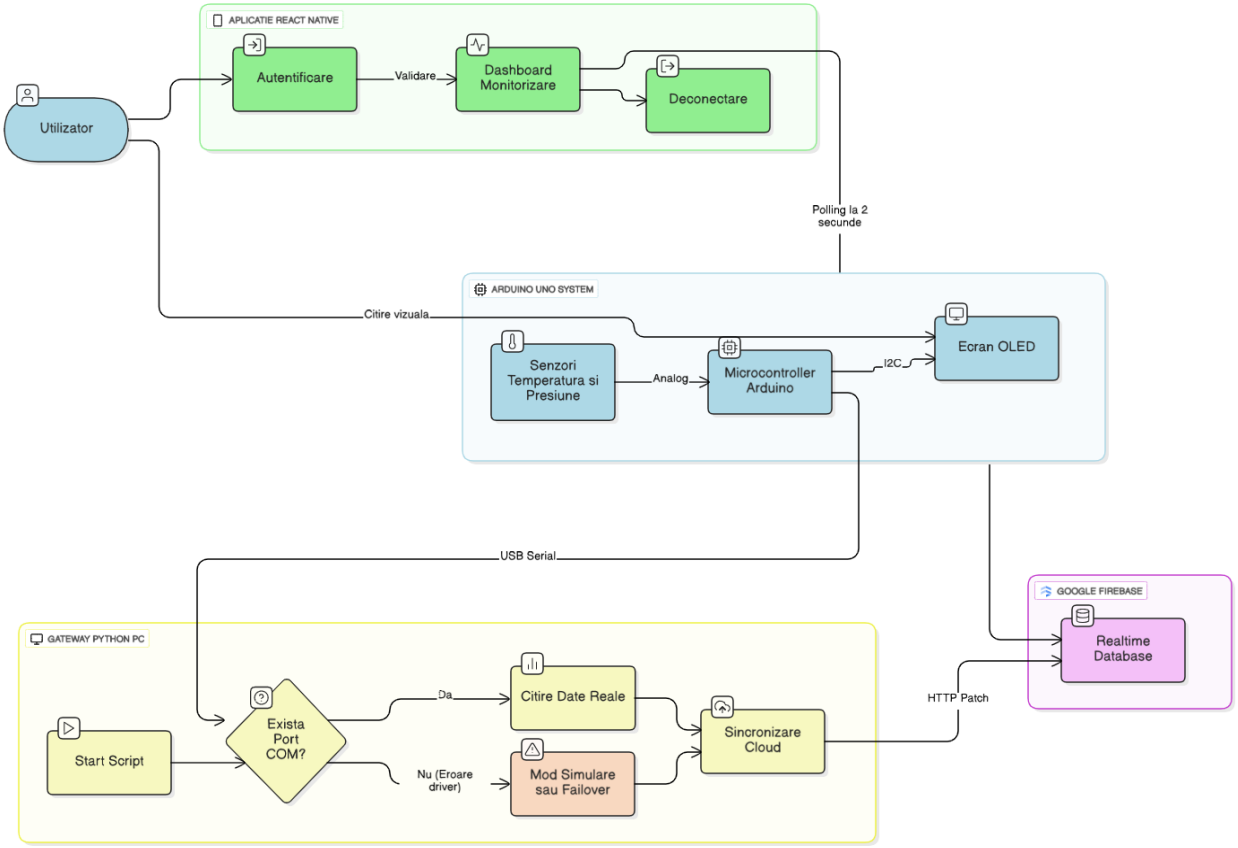
Obiectivele specifice sunt:

1. **Proiectarea nodului senzorial:** Interfațarea senzorilor analogici cu un microcontroler Arduino Uno și calibrarea acestora pentru o precizie ridicată.
2. **Dezvoltarea interfeței locale (HMI):** Afișarea datelor pe un ecran OLED pentru a asigura funcționalitatea sistemului în regim offline.
3. **Implementarea Gateway-ului IoT:** Realizarea unei aplicații middleware în Python care să preia datele seriale și să le transmită securizat către Cloud, având capacitatea de a funcționa în mod hibrid (Date Reale / Simulare).
4. **Stocarea și sincronizarea datelor:** Utilizarea platformei Google Firebase pentru stocarea datelor în format JSON și sincronizarea în timp real.
5. **Dezvoltarea aplicației mobile:** Crearea unei aplicații cross-platform (React Native) care să permită autentificarea utilizatorilor și vizualizarea grafică a parametrilor.

1.3 Specificații

Sistemul a fost proiectat respectând următoarele specificații tehnice și funcționale:

- **Unitate Centrală de Procesare:** Arduino Uno (ATmega328P), frecvență ceas 16MHz.
- **Senzorică:**
  - Temperatură: Termistor NTC 10kΩ, configurat în divizor de tensiune, domeniu de măsurare 0-100°C.
  - Presiune: Senzor analogic (mapare 0-1023 ADC la hPa).
- **Afișaj:** OLED 0.96 inch, rezoluție 128x64 pixeli, interfață I2C.
- **Conectivitate:**
  - Nivel Hardware-PC: Serial UART (USB), baud rate 9600 bps.
  - Nivel PC-Cloud: Protocol HTTP/REST, format date JSON.
- **Platformă Cloud:** Google Firebase Realtime Database (NoSQL).
- **Software Client:** Aplicație mobilă compatibilă Android/IOS, dezvoltată cu Expo și React Native.
- **Rată de eșantionare:** Achiziție și transmisie la fiecare 2-3 secunde.


APLICATIE REACT NATIVE
Utilizator
Autentificare
Validare
Dashboard Monitorizare
Deconectare
Polling la 2 secunde
ARDUINO UNO SYSTEM
Citire vizuala
Senzori Temperatura si Presiune
Analog
Microcontroller Arduino
I2C
Ecran OLED
USB Serial
GOOGLE FIREBASE
Realtime Database
GATEWAY PYTHON PC
Start Script
Exista Port COM?
Da
Nu (Eroare driver)
Citire Date Reale
Mod Simulare sau Failover
Sincronizare Cloud
HTTP Patch

2 Studiu bibliografic

2.1. Platforme hardware și unitatea de procesare

Elementul central al nodului senzorial este placa de dezvoltare Arduino Uno Rev3. Conform documentației oficiale a producătorului [1], aceasta este bazată pe microcontrolerul ATmega328P, o arhitectură pe 8 biți care operează la o frecvență de 16 MHz. Documentația tehnică specifică faptul că placa dispune de 6 pini de intrare analogică, esențiali pentru proiectul curent, întrucât permit citirea tensiunilor variabile generate de divizoarele rezistive ale senzorilor de temperatură și presiune. De asemenea, suportul nativ pentru protocoalele de comunicație UART și I2C, detaliat în specificațiile hardware [1], validează alegerea acestei platforme pentru interfațarea simultană cu un PC (Gateway) și un ecran OLED.

2.2. Modele matematice pentru achiziția datelor

Un aspect critic în monitorizarea parametrilor ambientali este precizia măsurării temperaturii folosind termistori NTC (Negative Temperature Coefficient). Deoarece relația dintre rezistența termistorului și temperatură este puternic neliniară, utilizarea unei aproximări liniare ar introduce erori semnificative.

Pentru liniarizarea răspunsului senzorului, s-a utilizat modelul matematic propus de J. Steinhart și S. R. Hart în studiul lor fundamental din 1968, *"Calibration curves for thermistors"* [2]. Ecuația Steinhart-Hart, definită ca o serie polinomială a logaritmului rezistenței, permite calcularea temperaturii absolute cu o precizie ridicată, fiind standardul în oceanografie și instrumentație de precizie. Implementarea acestui algoritm în firmware-ul microcontrolerului asigură o conversie corectă a valorilor ADC în grade Celsius.

2.3. Interfețe grafice embedded

Pentru interfața locală a dispozitivului (Human-Machine Interface - HMI), s-a optat pentru un afișaj OLED bazat pe driverul SSD1306. Controlul acestui periferic a fost realizat utilizând bibliotecile open-source dezvoltate de Adafruit Industries [3]. Biblioteca Adafruit_SSD1306 gestionează protocolul de comunicație la nivel jos (I2C) și memoria buffer a ecranului, în timp ce biblioteca Adafruit_GFX oferă primitive grafice abstracte. Această abordare modulară permite desenarea textului și a formelor geometrice prin manipularea directă a pixelilor, abstractizând complexitatea trimiterii comenzilor hexadecimale către controllerul ecranului [3].

2.4. Arhitecturi Cloud şi comunicaţii de date

Pentru componenta de stocare și sincronizare a datelor, s-a analizat platforma Google Firebase Realtime Database. Documentaţia API-ului REST [5] descrie mecanismul prin care dispozitivele externe pot interacţiona cu baza de date folosind cereri HTTP standard (GET, PUT, PATCH). În contextul acestui proiect, utilizarea metodei PATCH, documentată în ghidul oficial [5], permite actualizarea specifică a câmpurilor (temperatura, presiunea) fără a suprascrie întreaga structură de date, optimizând astfel lăţimea de bandă și viteza de procesare a Gateway-ului Python. Structura datelor este de tip JSON (JavaScript Object Notation), facilitând integrarea directă cu aplicaţiile web și mobile.

2.5. Dezvoltarea aplicaţiilor mobile

Componenta finală a sistemului, aplicaţia mobilă pentru monitorizare la distanţă, a fost dezvoltată utilizând framework-ul React Native. Conform documentaţiei oficiale [4], acesta permite construirea de interfeţe utilizator native folosind paradigme declarative și limbajul JavaScript. Componentele de bază (Core Components) precum View, Text sau Image [4] sunt mapate direct pe elementele UI native ale sistemelor de operare Android și iOS, asigurând performanţă și un aspect consistent.

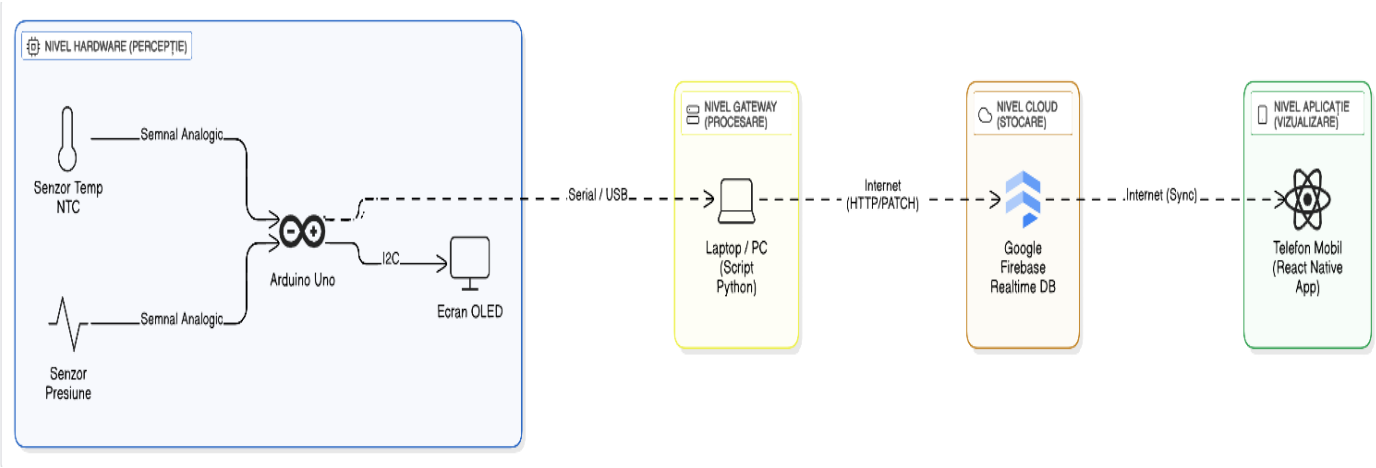
Pentru a accelera procesul de dezvoltare și testare, s-a utilizat setul de instrumente Expo. Documentaţia Expo [6] evidenţiază avantajele utilizării "Development Builds" și a sistemului de tunneling, care permit rularea și testarea aplicaţiei pe dispozitive fizice în timp real, fără a necesita compilarea complexă a codului nativ la fiecare modificare. Această tehnologie a facilitat iterarea rapidă a interfeţei grafice și integrarea cu baza de date Firebase.

3 Analiză, proiectare, implementare

- **3.1 Arhitectura Sistemului**

Sistemul este structurat pe patru nivele ierarhice, asigurând un flux de date unidirecțional de la senzor la utilizator:

1. **Nivelul Fizic (Senzorial):** Achiziția datelor brute.
2. **Nivelul Gateway (Procesare Locală):** Serializarea și transmiterea către internet.
3. **Nivelul Cloud (Stocare):** Persistența datelor.
4. **Nivelul Aplicație (Vizualizare):** Interfața cu utilizatorul.



- **3.2 Modelul matematic al achiziției de temperatură**

Pentru măsurarea temperaturii s-a utilizat un termistor de tip NTC (Negative Temperature Coefficient). Spre deosebire de senzorii digitali care livrează direct valoarea temperaturii, termistorii sunt componente rezistive pasive a căror rezistență scade odată cu creșterea temperaturii.

Caracteristica de transfer a unui termistor NTC este puternic neliniară, ceea ce face imposibilă utilizarea unei reguli de trei simplă pentru conversie. Pentru o determinare precisă a temperaturii, procesul de calcul este împărțit în două etape: determinarea rezistenței instantanee și aplicarea ecuației Steinhart-Hart.

- **3.2.1. Determinarea rezistenței electrice**

Termistorul este integrat într-un montaj de tip divizor de tensiune alături de o rezistență fixă ($Rref$) de 2kΩ. Microcontrolerul Arduino citește tensiunea la punctul median al divizorului prin Convertorul Analog-Digital (ADC) pe 10 biți.

Relația dintre valoarea digitală citită (ADCval) și rezistența curentă a termistorului (RNTC) este derivată din legea lui Ohm pentru divizorul de tensiune:

$$V_{out} = V_{in} \cdot \frac{R_{NTC}}{R_{NTC} + R_{fix}}$$

În implementarea software, calculul invers pentru aflarea rezistenței termistorului (RNTC) este realizat astfel:

$$R_{NTC} = \frac{R_{fix}}{(\frac{1023}{ADC_{val}} - 1)}$$

Unde:

- Rfix = 2000 Ω. (Rezistența cunoscută din circuit).
- 1023 = Rezoluția maximă a convertorului ADC (10 biți).
- ADCval = Valoarea brută citită de Arduino (0-1023).

- **3.2.2. Ecuația Steinhart-Hart**

Odată cunoscută rezistența RNTC, temperatura este calculată folosind Ecuația Steinhart-Hart. Aceasta este un model polinomial de ordinul trei, recunoscut în industrie pentru precizia sa ridicată în aproximarea relației rezistență-temperatură a semiconductorilor.

Forma generală a ecuației este:

$$\frac{1}{T} = A + B\ln(R_{NTC}) + C(\ln(R_{NTC}))^3$$

Unde:

- **T**= Temperatura absolută în grade Kelvin (K).
- **RNTC** = Rezistența termistorului în Ohmi (Ω), calculată la pasul anterior.
- **A, B, C** = Coeficienții specifici termistorului, determinați experimental sau furnizați de producător.

Pentru senzorul utilizat în acest proiect, coeficienții implementați în cod sunt:

$$A = 1.234166184 \times 10^{-3}$$

$$B = 2.712699866 \times 10^{-4}$$

- $$C = 0.928986247 \times 10^{-7}$$

- **3.2.3. Conversia finală și calibrarea**

Deoarece ecuația furnizează rezultatul în grade Kelvin, ultimul pas al algoritmului este conversia în grade Celsius și aplicarea unui factor de corecție (offset) pentru compensarea erorilor de auto-încălzire ale senzorului:

$$T_{Celsius} = T_{Kelvin} - 273.15 - T_{offset}$$

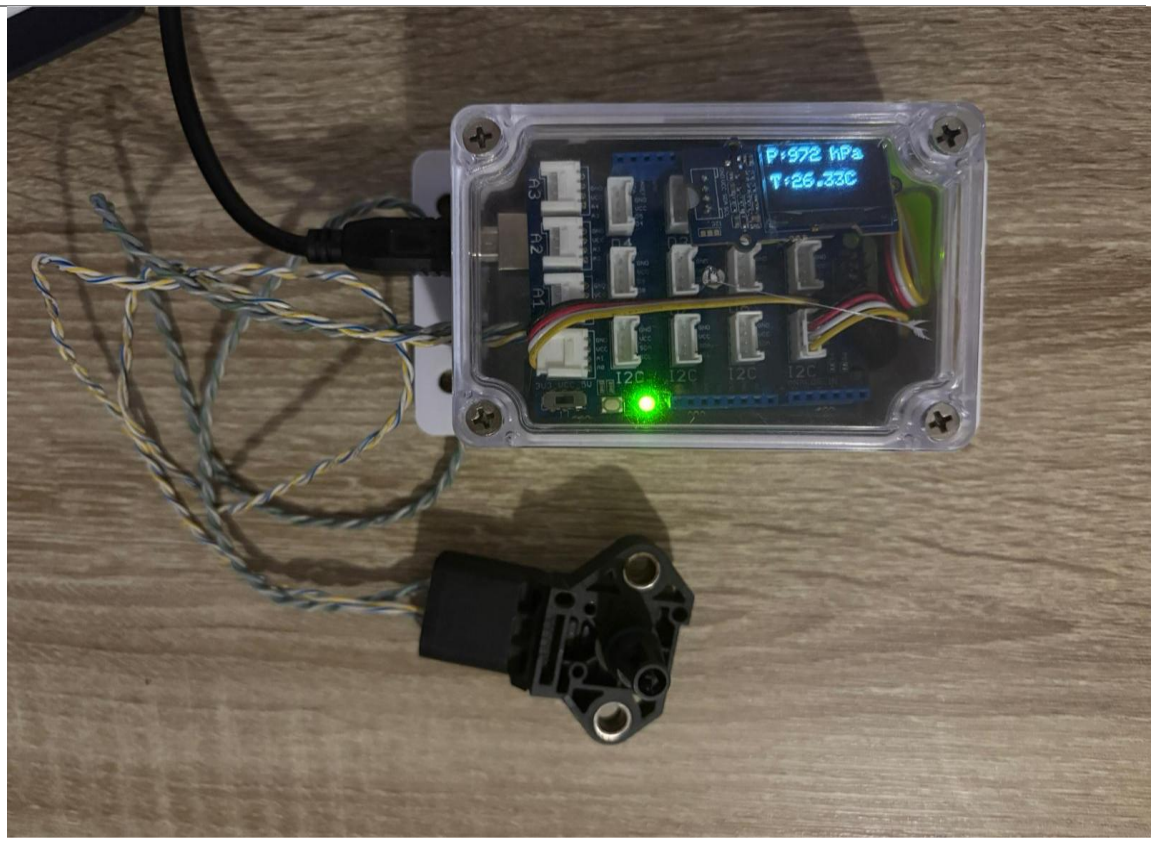
În cadrul testelor efectuate, s-a stabilit experimental un offset de 2.3 $°C$ pentru a alinia valorile citite cu cele ale unui termometru etalon.

3.3 Implementare Hardware

Schema electrică se bazează pe placa de dezvoltare Arduino Uno.

- **Circuitul de temperatură:** Este realizat un divizor de tensiune format dintr-un termistor NTC (conectat la VCC) și o rezistență de 2kΩ (conectată la GND). Punctul median este conectat la pinul analogic A1.
- **Circuitul de presiune:** Senzorul analogic este conectat la pinul A0.
- **Afișaj:** Ecranul OLED comunică pe pinii dedicați I2C (A4 - SDA, A5 - SCL).

Întregul montaj a fost integrat într-o carcasă transparentă, prevăzută cu decupaje pentru accesul la portul USB, asigurând protecția mecanică.



- **3.4 Implementare Software**
- **3.4.1 Firmware (Arduino C++)**

Codul încărcat pe microcontroler realizează următoarele funcţii ciclice (în void loop):

1. **Citire ADC:** Preia valorile digitale (0-1023) de la senzori.
2. **Calcul Matematic:** Aplică formula Steinhart-Hart pentru temperatură.
3. **Calibrare Software:** S-a implementat un offset de corecție (-2.3 grade) pentru a compensa toleranţa componentelor şi auto-încălzirea senzorului.
4. **Afişare:** Utilizează biblioteca Adafruit_GFX pentru desenarea pe ecranul OLED.
5. **Serializare:** Formatează datele într-un string CSV (temperatura,presiune) şi le trimite pe portul USB.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

int offset = 100; // low pressure calibration
int span = 929;   // high pressure calibration
float Vin = 4.85;   // 4.8 volts - USB powered
float R1 = 2000;   // Temperature resistor value

float A = 1.234166184e-03, B = 2.712699866e-04, C = 0.9289862476e-07; // Steinhart-Hart and Hart Coefficients

void setup() {
  Serial.begin(9600); // Pornim comunicarea cu Laptopul

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    for (;;);
  }
}

void loop() {
  // --- CALCUL PRESIUNE ---
  float pressure = map(analogRead(A0), offset, span, 500, 4000);
  int pressureInt = int(pressure); // Convertim la numar intreg
```

```
// --- CALCUL TEMPERATURA ---
int temperatureVoltage = analogRead(A1);
float temperatureBuffer = temperatureVoltage * Vin;
float Vout = (temperatureBuffer) / 1024.0;
temperatureBuffer = (Vin / Vout) - 1;
float R2 = R1 / temperatureBuffer;

float logR2 = log(R2);
float temperature = (1.0 / (A + B * logR2 + C * logR2 * logR2 * logR2));
temperature -= 273.15; // Convert to Celsius
temperature= temperature-5;

// 1. AFISARE PE ECRAN
display.clearDisplay();
display.setTextSize(2);
display.setTextColor(SSD1306_WHITE);

display.setCursor(0, 0);
display.print(F("P:"));
display.print(pressureInt);
display.print(F(" hPa"));

display.setCursor(0, 30);
display.print(F("T:"));
display.print(temperature);
display.print(F("C"));

display.display();

//2. TRIMITERE CATRE PYTHON (Format: "24.50,1002") ---
```

```
  Serial.print(temperature);
  Serial.print(",");
  Serial.println(pressureInt);

  delay(2000); // Asteapta 2 secunde
}
```

- **3.4.2 Gateway (Python)**

Scriptul Python reprezintă "podul" dintre hardware şi internet. Acesta rulează pe un PC şi include mecanisme de robusteţe:

- **Modul Hibrid:** Scriptul poate funcţiona în modul FOLOSESTE_SENZOR_REAL = True (citire USB) sau False (generare date aleatoare). Acest lucru asigură testarea aplicaţiei mobile chiar şi în lipsa hardware-ului.

```
import time
import requests
import json
import random
import serial
import sys

FOLOSESTE_SENZOR_REAL = True

PORT_SERIAL = "COM5"
VITEZA_SERIALA = 9600

FIREBASE_URL = "https://termostatrc-default-rtdb.europe-
west1.firebasedatabase.app/date.json"

print("PORNIRE SISTEM MONITORIZARE IoT")
if FOLOSESTE_SENZOR_REAL:
    print(f"MOD: CITIRE SENZOR REAL pe {PORT_SERIAL}")
else:
    print("MOD: SIMULARE DATE (Fara cablu)")
print("-------------------------------------------------")

```

```
# Functie pentru conectare la Arduino
def conectare_arduino():
    if not FOLOSESTE_SENZOR_REAL:
        return None
    try:
        ser = serial.Serial(PORT_SERIAL, VITEZA_SERIALA, timeout=1)
        time.sleep(2)
        print("Conexiune Seriala Reusita!")
        return ser
    except Exception as e:
        print(f"EROARE: Nu pot conecta Arduino pe {PORT_SERIAL}")
        print("   Verifica daca ai cablul bun si daca ai pus COM-ul
corect.")
        sys.exit()

ser = conectare_arduino()

while True:
    try:
        temp_final = 0.0
        pres_final = 0
        status_curent = ""

        if FOLOSESTE_SENZOR_REAL:
            if ser.in_waiting > 0:
                line = ser.readline().decode('utf-8').strip()
                if "," in line:
                    parts = line.split(",")
                    if len(parts) == 2:
                        temp_final = float(parts[0])
                        pres_final = int(parts[1])
                        status_curent = "Online (Senzor)"
                else:
                    continue
            else:
                continue

        else:
            temp_final = round(random.uniform(23.0, 26.0), 2)
            pres_final = random.randint(980, 1005)
            status_curent = "Online (Simulat)"
            time.sleep(3) # Pauza doar la simulare

        #TRIMITEREA CATRE FIREBASE
        if temp_final != 0:
```

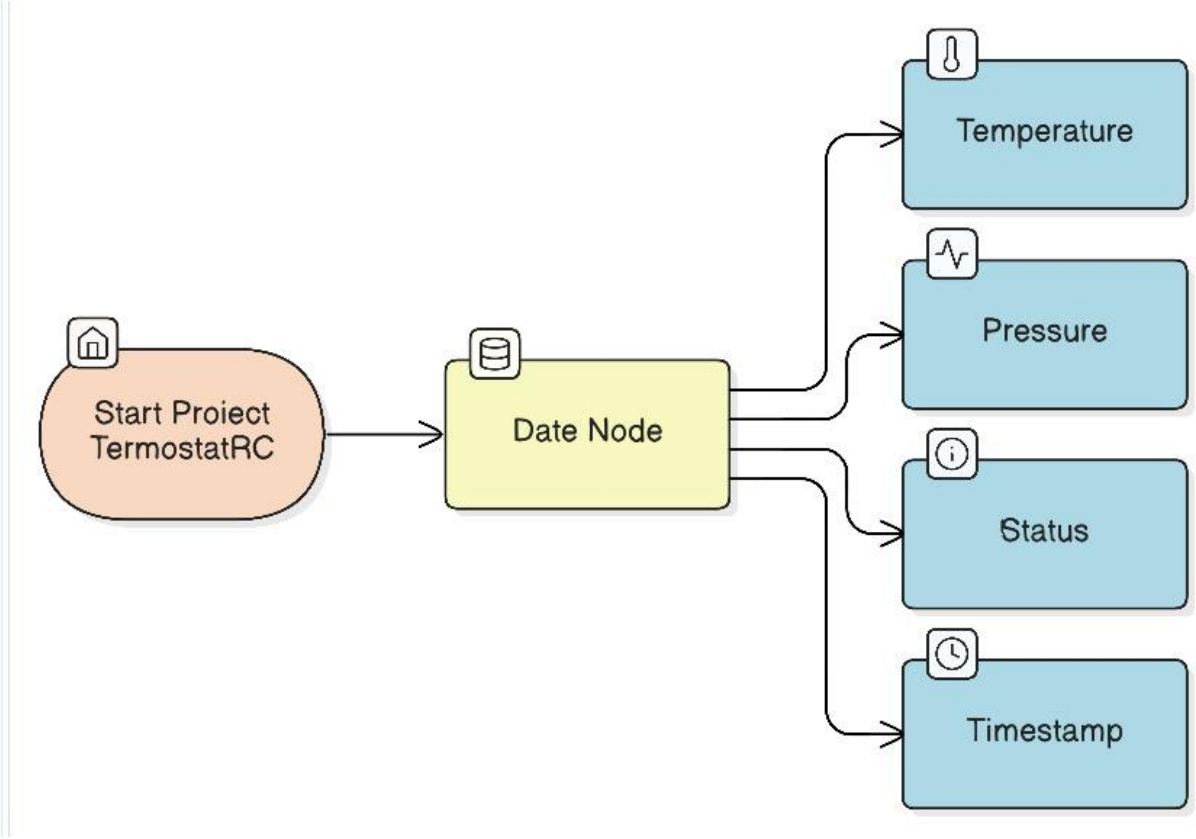
```
            data_to_send = {
                "temperatura": temp_final,
                "presiune": pres_final,
                "status": status_curent,
                "timestamp": time.time()
            }
            response = requests.patch(FIREBASE_URL, json=data_to_send)

            if response.status_code == 200:
                print(f"Trimis: Temp={temp_final} C | Pres={pres_final}
hPa | Mod: {status_curent}")
            else:
                print(f"Eroare Firebase: {response.status_code}")

    except ValueError:
        print("Date corupte de la senzor, ignor...")
    except KeyboardInterrupt:
        print("\nOprit de utilizator.")
        break
    except Exception as e:
        print(f"Eroare: {e}")
        if FOLOSESTE_SENZOR_REAL:
            time.sleep(1)
```

- **Parsare şi Transmisie:** Datele primite pe serială sunt transformate în dicţionar Python şi trimise ca JSON către Firebase folosind biblioteca requests.

```
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=977 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=24.08 C | Pres=981 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=24.08 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=24.08 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=23.97 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=24.08 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=24.08 C | Pres=985 hPa | Mod: Online (Senzor)
📤 Trimis: Temp=24.08 C | Pres=985 hPa | Mod: Online (Senzor)
```

https://termostatrc-default-rtdb.europe-west1.firebasedatabase.app

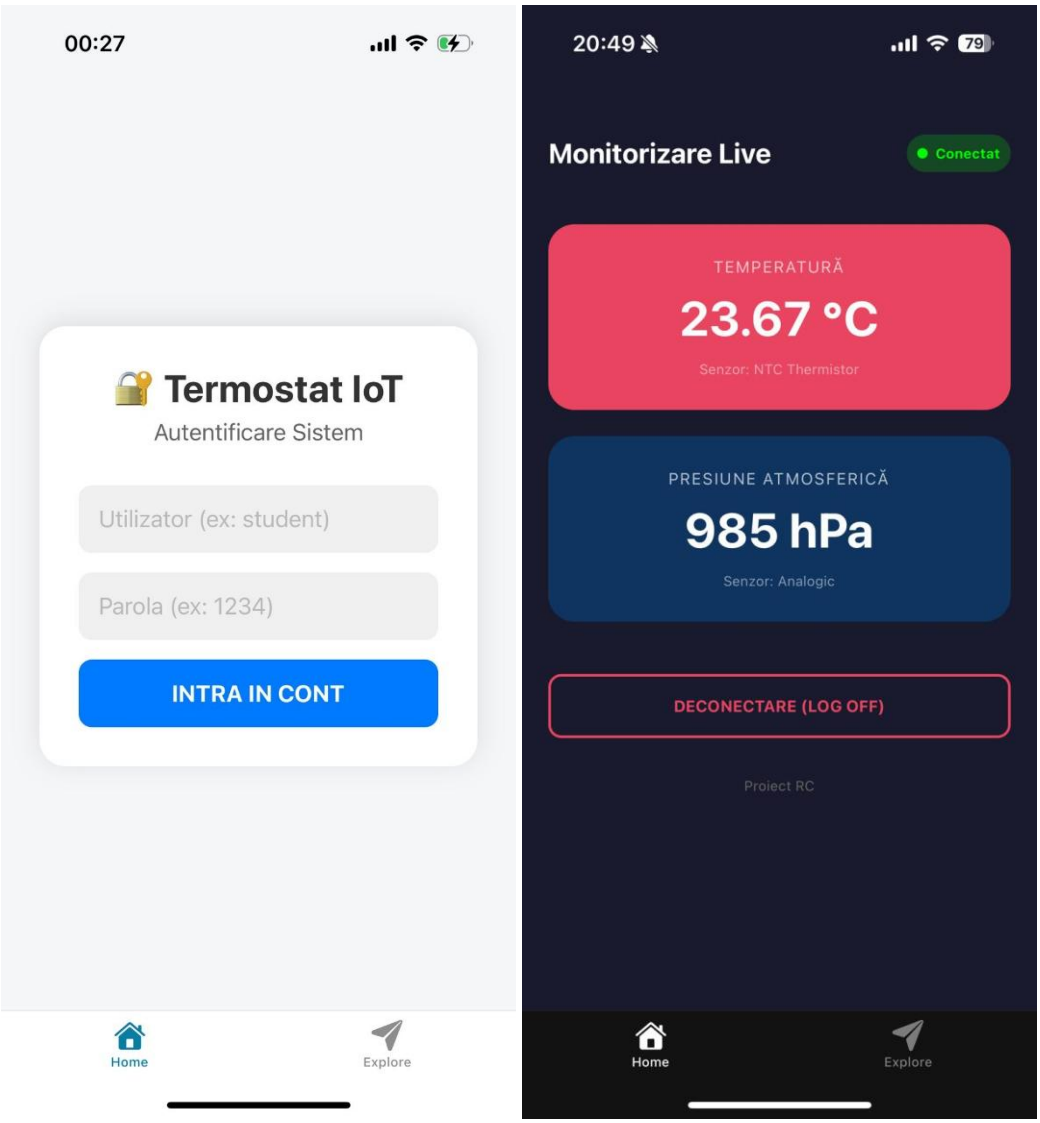
https://termostatrc-default-rtdb.europe-west1.firebasedatabase.app/
date
presiune: 985
status: "Online (Senzor)"
temperatura: 24.08
timestamp: 1765651553.7061384



- **3.4.3 Aplicaţia Mobilă (React Native)**

Aplicaţia a fost dezvoltată modular:

- **Ecranul de Login:** Implementează o verificare simplă a credenţialelor (Hardcoded pentru prototip: user student).
- **Dashboard:** Utilizează React Hooks (useState, useEffect) pentru a gestiona starea aplicaţiei. Funcţia de actualizare apelează baza de date la intervale regulate (Polling la 2 secunde) pentru a reflecta schimbările în timp real. Interfaţa grafică este stilizată folosind StyleSheet, oferind indicatori vizuali colorați pentru temperatură (roșu) și presiune (albastru).



4 Concluzii

4.1 Rezultate obținute

Proiectul a fost finalizat cu succes, rezultând un prototip funcțional care îndeplinește toate cerințele caietului de sarcini.

- Sistemul realizează citiri precise, validate prin comparație cu termometre comerciale, grație calibrării software implementate.
- Latența totală a sistemului (timpul scurs de la variația temperaturii până la afișarea pe telefon) este sub 2 secunde.
- Implementarea modului hibrid în Python a demonstrat utilitatea în scenarii de debugging și prezentare, asigurând continuitatea funcționării.

4.2 Direcții de dezvoltare

Pentru evoluția ulterioară a proiectului, se propun următoarele îmbunătățiri:

1. **Eliminarea PC-ului Gateway:** Migrarea codului pe un microcontroler cu Wi-Fi integrat (ESP32 sau ESP8266) pentru a comunica direct cu Firebase, făcând dispozitivul complet independent.
2. **Notificări Push:** Integrarea serviciilor de notificare pentru a alerta utilizatorul când temperatura depășește un prag critic (ex: pericol de îngheț).
3. **Istoric și Grafice:** Extinderea aplicației mobile pentru a stoca istoricul datelor și a afișa grafice de evoluție pe ultimele 24 de ore.

5. Bibliografie

[1] Arduino, "Arduino Uno Rev3 Documentation", [Online]. Disponibil: https://docs.arduino.cc/hardware/uno-rev3/

[2] J. Steinhart and S. R. Hart, "Calibration curves for thermistors," Deep Sea Research and Oceanographic Abstracts, vol. 15, no. 4, pp. 497-503, 1968.

[3] Adafruit Industries, "Adafruit SSD1306 Library & GFX Library", GitHub, [Online]. Disponibil: https://github.com/adafruit/Adafruit_SSD1306

[4] Meta Open Source, "React Native Documentation - Core Components", [Online]. Disponibil: https://reactnative.dev/docs/intro

[5] Google, "Firebase Realtime Database REST API Guide", [Online]. Disponibil: https://firebase.google.com/docs/database/rest/start

[6] Expo, "Expo Documentation - Development Builds", [Online]. Disponibil: https://docs.expo.dev/